

1. Beadandó feladat dokumentáció

Készítette:

Kecskés Norbert Dániel

E-mail:

e2a4og@inf.elte.hu

Feladat:

Készítsünk programot, amellyel a következő játékot játszhatjuk.

Adott egy $n \times n$ mezőből álló játékpálya, amelyben egy elszabadult robot bolyong, és a feladatunk az, hogy beteretljük a pálya közepén található mágnes alá, és így elkapjuk.

A robot véletlenszerű pozícióban kezd, és adott időközönként lép egy mezőt (vízszintesen, vagy függőlegesen) úgy, hogy általában folyamatosan előre halad egészen addig, amíg falba nem ütközik. Ekkor véletlenszerűen választ egy új irányt, és arra halad tovább. Időnként még jobban megkegyül, és akkor is irányt vált, amikor nem ütközik falba.

A játékos a robot terelését úgy hajthatja végre, hogy egy mezőt kiválasztva falat emelhet rá. A felhúzott falak azonban nem túl strapabíróak. Ha a robot ütközik a fallal, akkor az utána eldőlt. A ledőlt falakat már nem lehet újra felhúzni, ott a robot később akadály nélkül áthaladhat.

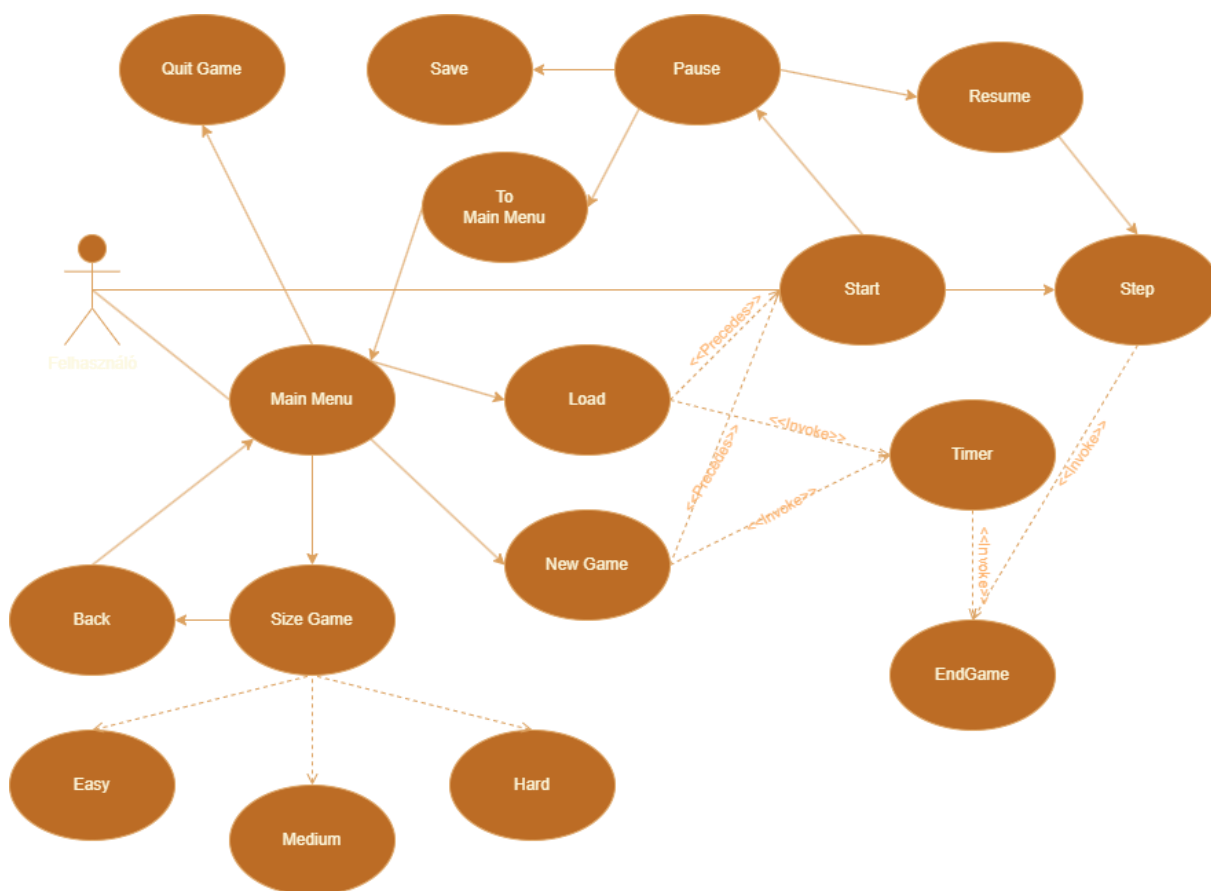
A program biztosítson lehetőséget új játék kezdésére a pályaméret megadásával (7×7 , 11×11 , 15×15), valamint játék szüneteltetésére (ekkor nem telik az idő, nem lép a robot, és nem lehet mezőt se kiválasztani). Ismerje fel, ha vége a játéknak, és jelenítse meg, hogy milyen idővel győzött a játékos. A program játék közben folyamatosan jelezze ki a játékidőt. Ezen felül szüneteltetés alatt legyen lehetőség a játék elmentésére, valamint betöltésére.

Elemzés:

- A játékot három nehézségi szinttel játszhatjuk: könnyű (7×7 , 49 generált mező), közepes (11×11 , 121 előre generált mező), nehéz (15×15 , 225 előre generált mező). A játékot nem lehet elindítani, nehézségi szint állítása nélkül.
- A feladatot egyablakos asztali alkalmazásként Windows Forms grafikus felülettel valósítjuk meg.
- Az ablakban elhelyezünk egy menüt a következő menüpontokkal: New Game (Új játék), Load (Játék betöltése), Size (Méret állítása (Easy, Medium, Hard, Back)). Az ablak alján megjelenítünk egy státuszsort, amely a játéktér méretét jelzi.
- A játéktáblát nehézségi szinttől függően egy 7×7 , 11×11 , 15×15 nyomógomb álló rács reprezentálja. A nyomógomb egérekattintás hatására egy „falat” tesz le az adott mezőre megváltoztatva a színét. Az így megváltoztatott mezőt a játék hátralévő idejében nem lehet megváltoztatni. A táblán az előre generált mezőt nem engedjük megváltoztatni, ez a játéktér közepe, amit piros színnel van jelölve.

A játék automatikusan feldob egy dialógusablakot, amikor vége a játéknak (nyertél vagy veszítettél). Szintén dialógusablakokkal végezzük el a mentést, illetve betöltést, a fájlneveket a felhasználó adja meg.

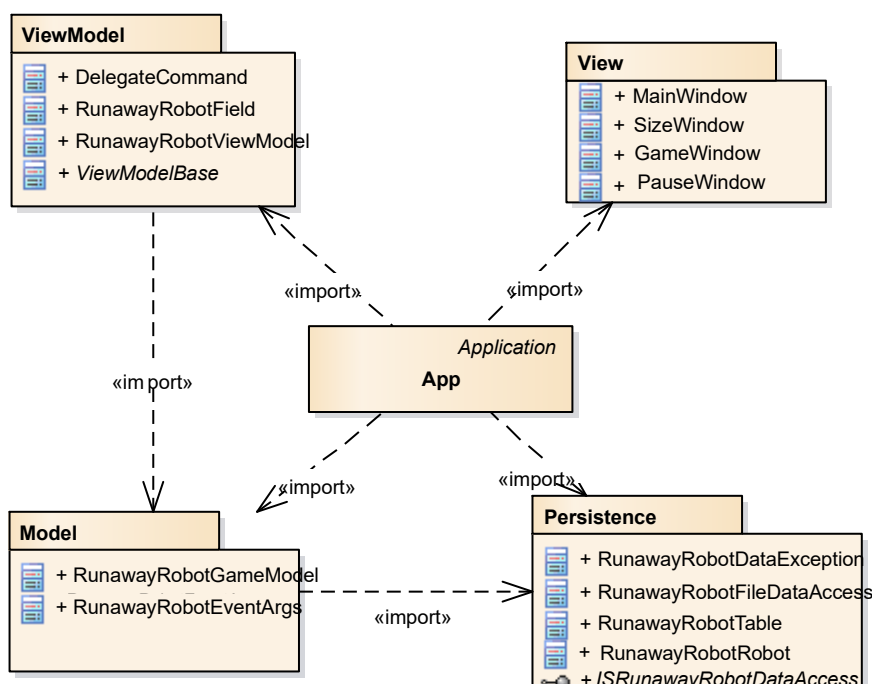
- A felhasználói esetek az 1. ábrán láthatóak.



1. ábra: Felhasználói esetek diagramja

Tervezés:

- Programszerkezet:
- A programot MVVM architektúrában valósítjuk meg, ennek megfelelően **View**, **Model**, **ViewModel** és **Persistence** névttereket valósítunk meg az alkalmazáson belül. A program környezetét az alkalmazás osztály (**App**) végzi, amely példányosítja a modellt, a nézetmodellt és a nézetet, biztosítja a kommunikációt, valamint felügyeli az adatkezelést. A program csomagszerkezete a 2. ábrán látható.
- A program szerkezetét két projektre osztjuk implementációs megfontolásból: a **Persistence** és **Model** csomagok a program felületfüggetlen projektjében, míg a **ViewModel** és **View** csomagok a WPF függő projektjében kapnak helyet.



Perzisztencia:

- Az adatkezelés feladata a RunawayRobot táblával és robottal kapcsolatos információk tárolása, valamint a betöltés/mentés biztosítása.
- A **RunawayRobotTable** osztály egy érvényes RunawayRobot táblát biztosít (azaz mindig ellenőrzi a beállított értékeket), ahol minden mezőre ismert az értéke (**FieldValue**), illetve a zároltsága (**Locked**). Utóbbit a játék kezdetekor generált, illetve értékekre alkalmazzuk (**GetFieldValue**, **IsLocked**), illetve direkt beállítás (**SetFieldValue**, **LockField**, **UnLockField**) elvégzésére.
- A **RunawayRobotRobot** osztály egy érvényes RunawayRobot robotot biztosít (azaz mindig ellenőrzi a beállított értékeket), ahol minden mezőre ismert az értéke (**direction**), illetve a robot koordinátáit (**robotX**, **robotY**). Utóbbit a játék kezdetekor generált, illetve értékekre alkalmazzuk (**Move**, **Occupied**), illetve direkt beállítás (**SetX**, **SetY**, **SetDirection**) elvégzésére.
- A hosszú távú adattárolás lehetőségeit az **IRunawayRobotDataAccess** interfész adja meg, amely lehetőséget ad a tábla és a robot betöltésére (**LoadAsync**), valamint mentésére (**SaveAsync**). A műveleteket hatékonysági okokból aszinkron módon valósítjuk meg.

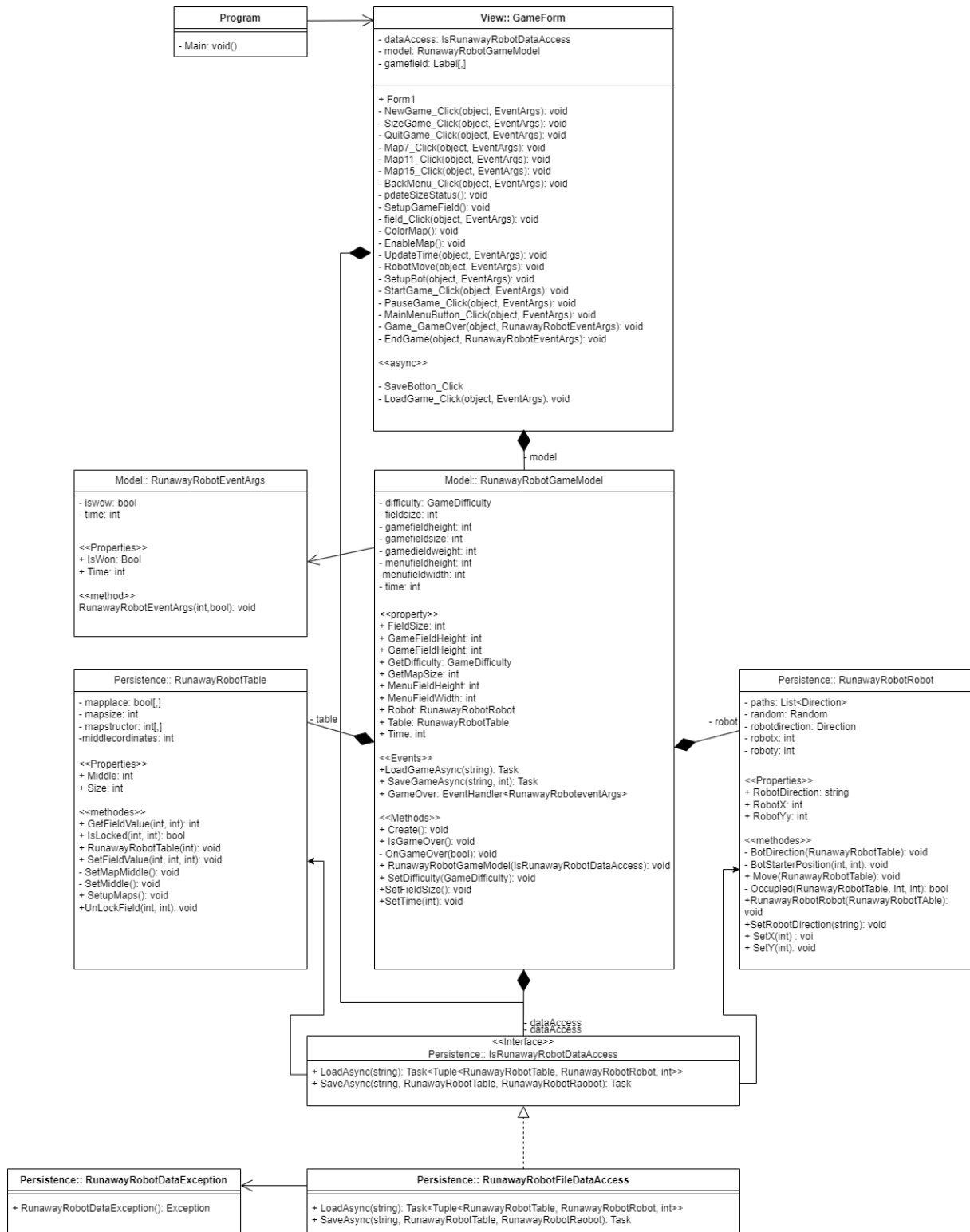
- Az interfészt szöveges fájl alapú adatkezelésre a **RunawayRobotFileDataAccess** osztály valósítja meg. A fájlkezelés során fellépő hibákat a **RunawayRobotDataException** kivétel jelzi.
- A program az adatokat szöveges fájlként tudja eltárolni, melyek az **stl** kiterjesztést kapják. Ezeket az adatokat a programban fő menüjében lehet betölteni, illetve a játék megállítása során ki lehet menteni az aktuális állást.
- A fájl első sora megadja az eltelt időt és a tábla méretét. A fájl második sora pedig a robot koordinátáit és az irányát tartalmazza. A fájl többi része izomorf leképezése a játéktáblának, azaz összesen 7,11,15 sor következik, és minden sor 7,11,15 számot tartalmaz szóközzel választva. A számok 0-5 közöttiek lehetnek, ahol 0 reprezentálja a még üres mezőt.

2. ábra: Az alkalmazás csomagdiagramja

Modell:

- A modell lényegi részét a **RunawayRobotGameModel** osztály valósítja meg, amely szabályozza a tábla tevékenységeit, valamint a játék egyéb paramétereit, úgymint az idő (**time**). A típus lehetőséget ad új játék kezdésére (**NewGame**), valamint mozgásra (**RobotMove**). Új játéknál megadható a kiinduló játéktábla is, különben automatikusan felugrik egy hiba ablak. Az idő előreléptetését időbeli lépések végzéséve (**SetTime**) tehetjük meg.
- A játékállapot megváltozásáról a játék végéről a **GameOver** esemény tájékoztat. Az események argumentuma (**RunawayRobotEventArgs**) tárolja a győzelem állapotát, valamint a játékidőt.
- A modell példányosításkor megkapja az adatkezelés felületét, amelynek segítségével lehetőséget ad betöltésre (**LoadGameAsync**) és mentésre (**SaveGameAsync**)
- A játék nehézségét a **GameDifficulty** felsorolási típuson át kezeljük, és a **GameModel** osztályban konstansok segítségével tároljuk az egyes nehézségek paramétereit.
- Nézet:
- A nézetet a **GameForm** osztály biztosítja, amely tárolja a modell egy példányát (**model**), valamint az adatelérés konkrét példányát (**dataAccess**).
- A játéktáblát egy dinamikusan létrehozott címke mátrix (**label**) reprezentálja. A felületen létrehozunk a megfelelő menüpontokat, illetve

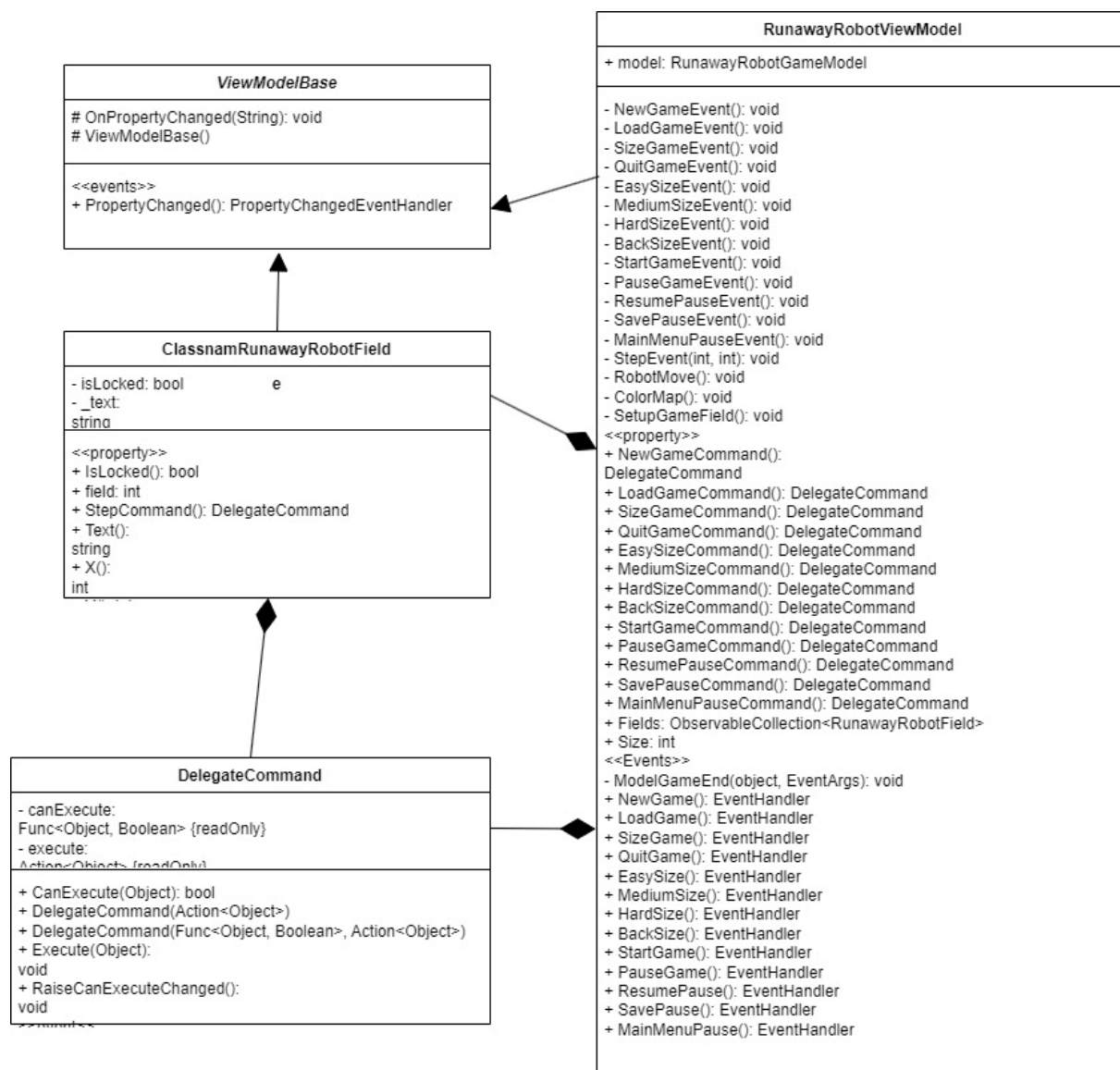
- státuszsort, valamint dialógusablakokat, és a hozzájuk tartozó eseménykezelőket. A játéktábla generálását (**SetupGameField**), illetve a robot beállítását (**SetupBot**) külön metódusok végzik.
- A játék időbeli kezelését egy időzítő végzi (**timer**), amelyet mindig aktiválunk játék során, illetve inaktiválunk, amennyiben bizonyos menüfunkciók futnak.
- A program teljes statikus szerkezete a 3. ábrán látható.



3. ábra: A Model csomag osztálydiagramja

- Nézetmodell:
- A nézetmodell megvalósításához felhasználtunk egy általános utasítás (**DelegateCommand**), valamint egy ős változásjelző (**ViewModelBase**) osztályt.

- A nézetmodell feladatait a **RunawayRobotViewModel** osztály látja el, amely parancsokat biztosít az új játék kezdéséhez, játék betöltéséhez, mentéséhez, valamint a kilépéshez. A parancsokhoz eseményeket kötünk, amelyek a parancs lefutását jelzik a vezérlőnek. A nézetmodell tárolja a modell egy hivatkozását (**model**), de csupán információkat kér le tőle, illetve a játéknehézséget szabályozza. Direkt nem avatkozik a játék futtatásába.
- A játékmező számára egy külön mezőt biztosítunk (**RunawayRobotField**), amely eltárolja a pozíciót, szöveget, engedélyezettséget, valamint a lépés parancsát (**StepCommand**). A mezőket egy felügyelt gyűjteménybe helyezzük a nézetmodellbe (**Fields**).



4. ábra: A nézetmodell osztálydiagramja

- Nézet:

- A nézet csak négy képernyőt tartalmaz, a **MainWindow** osztályt a **SizeWindow** osztályt a **GameWindow** osztályt és a **PauseWindow** osztályt. A nézet egy rácsban tárolja a játékmegőzt, és a gombokat. A játékmegőzt egy **ItemsControl** vezérlő, ahol dinamikusan felépítünk egy rácsot (**UniformGrid**), amely gombokból áll. Minden adatot adatkötéssel kapcsolunk a felülethez, továbbá azon keresztül szabályozzuk a gombok színét is.
- A fájlnev bekérését betöltéskor és mentéskor, valamint a figyelmeztető üzenetek megjelenését beépített dialógusablakok segítségével végezzük.
- Környezet:
- Az **App** osztály feladata az egyes rétegek példányosítása (**AppStartup**), összekötése, a nézetmodell, valamint a modell eseményeinek lekezelése, és ezáltal a játék, az adatkezelés, valamint a nézetek szabályozása.
- A játék léptetéséhez tárol egy időzítőt is (**timer**), amelynek állítását is szabályozza az egyes funkciók hatására.

App
- model: RunawayRobotGameModel - viewMain: MainWindow - viewSize: SizeWindow - viewGame: GameWindow - viewPause: PauseWindow - viewModel: RunawayRobotViewModel
+ App - AppStartup(object, StartupEventArgs): void - TimerTick(object, EventArgs): void - UpdateTime: void - ViewModelNewGame(object, EventArgs): void - ViewModelLoadGame(object, EventArgs): void - ViewModelSizeGame(object, EventArgs): void - ViewModelQuitGame(object, EventArgs): void - ViewModelEasySize(object, EventArgs): void - ViewModelMediumSize(object, EventArgs): void - ViewModelHardSize(object, EventArgs): void - ViewModelBackSize(object, EventArgs): void - UpdateSize: void - ViewModelStartGame(object, EventArgs): void - ViewModelPauseGame(object, EventArgs): void - ViewModelResumePause(object, EventArgs): void - ViewModelSaveGame(object, EventArgs): void - ViewModelMainMenuPause(object, EventArgs): void - ModelGameOver(object, EventArgs): void

5. ábra: Az alkalmazás osztálydiagramja

Tesztelés:

- A modell funkcionalitása egységtesztek segítségével lett ellenőrizve a

RunawayRobotGameModelTest osztályban.

- Az alábbi tesztesetek kerültek megvalósításra:
- **RunawayRobotGameModelNewGameEasyTest**,
RunawayRobotGameModelNewGameMediumTest,
RunawayRobotGameModelNewGameHardTest: Új játék indítása, a mezők kitöltése, valamint a nehézség ellenőrzése a nehézségi fokozat függvényében.
- **RunawayRobotGameModelMoveTest**: Játékbeli robot mozgásának hatásainak ellenőrzése, játék megkezdése előtt, valamint után.
- **RunawayRobotGameModelLoadTest**: A játék modell betöltésének tesztelése mockolt perzisztencia réteggel.